

Bangladesh University of Business and Technology
Department of Computer Science and Engineering

CSE 400 : Software Development Project IV

PROJECT REPORT

Semester: Summer 2026

Title: Nikah — Islamic Nikah Matrimony Platform

Submission Date: August 2026

TEAM MEMBERS

Full ID	Full Name	Intake/Section
22235103137	Md. Mahamudul Hasan	51/11
22235103251	Sumya Soma	51/11
22235103411	Salat Mahmud Chowdhury	51/11
22235103680	Alfa Sany	51/11
22235103321	Zinat Parvin	51/11

SUPERVISED BY

Ahmed Shafkat

Assistant Professor, Department of CSE
Bangladesh University of Business and Technology (BUBT)

Abstract

Nikah is a full-stack, Islamic matrimony web platform designed for the Bangladeshi Muslim community. It provides a private, secure and religion-aligned environment in which individuals can build detailed *biodata* (matrimonial profile) records, search for compatible partners, communicate through verifiable channels, and progress towards *Nikah* — the Islamic marriage contract. The presentation tier is a single-page application built with **React 18** and **Vite**, styled with **Tailwind CSS** and the **shadcn/ui** component system (new-york style, built on Radix UI), themed with a custom emerald-and-gold design token set. The application tier is a **Node.js / Express 5** REST API, and the data tier is **MongoDB Atlas** managed through the **Mongoose 9** ODM across thirteen collections. Authentication is hybrid: regular members authenticate through **Firebase** (email/password and Google) and exchange their verified identity for a short-lived server JSON Web Token (JWT), while a **dedicated administrator** authenticates against environment-configured credentials issued by the server itself — completely separate from Firebase. Role-based access control (RBAC) distinguishes three tiers: regular user, premium subscriber, and administrator. The platform delivers fifteen functional modules including biodata management, filtered search, a compatibility matching engine, contact-request workflow with payment gating, favourites, messaging, notifications, profile-view tracking, recently-viewed history, premium subscriptions, success stories, a reporting system, and a rich analytics-driven admin dashboard. The interface is bilingual (English / Bangla), fully responsive, ships with a dark mode, and is deployed on free-tier cloud infrastructure — the API on Vercel, the front end on Netlify, and the database on MongoDB Atlas.

Keywords: Islamic matrimony, MERN stack, React, Express, MongoDB, Firebase, JWT, role-based access control, shadcn/ui, responsive web design, internationalization, Bangladesh.

Acknowledgment

We express our sincere gratitude to our supervisor, **Mr. Ahmed Shafkat**, Assistant Professor, Department of Computer Science and Engineering, Bangladesh University of Business and Technology (BUBT), for his continuous guidance, valuable suggestions and encouragement throughout the development of this project. We are also thankful to the faculty members of the Department of CSE for their support and to the open-source community whose libraries and documentation made this work possible. Finally, we thank our families for their patience and encouragement during the entire Software Development Project IV cycle.

Table of Contents

Abstract	i
Acknowledgment	ii
List of Figures	v
List of Tables	vi
1 Introduction	1
1.1 Problem Specification	1
1.2 Objectives	1
1.3 Scope	2
1.4 Flow of the Project	2
1.5 Organization of the Project Report	3
2 Background and Related Works	4
2.1 Existing System Analysis	4
2.2 Supporting Literatures	5
2.3 Problem Statement and Research Gap	5
3 System Analysis and Design	6
3.1 Technology and Tools	6
3.1.1 Front-End Stack	6
3.1.2 Back-End Stack	7
3.1.3 Database	7
3.1.4 Tools Used	8
3.2 Development Model	8
3.3 System Architecture	8
3.4 Use-Case Analysis	9
3.5 Context-Level Diagram (DFD-0)	9
3.6 Data-Flow Diagram (DFD-1)	10
3.7 Database Schema	11
3.8 Algorithms and Flowcharts	12
3.8.1 Compatibility Matching	12
3.8.2 Authentication Flow	14
3.8.3 Contact-Request Workflow	15

4	Implementation	16
4.1	Front-End / Interface Design	16
4.2	Back-End	17
4.3	Modules and Features	18
5	User Manual	20
5.1	System Requirements	20
5.1.1	Hardware Requirements	20
5.1.2	Software Requirements	20
5.2	User Interfaces	20
5.2.1	Panel A — Public Pages	20
5.2.2	Panel B — User Dashboard	22
5.2.3	Panel C — Admin Panel	22
5.2.4	Login Credentials	23
5.3	Getting Started	23
6	Conclusion	24
6.1	Conclusion	24
6.2	Limitations	24
6.3	Future Works	25
	References	26
A	Glossary and Supplementary Material	28
A.1	Glossary of Terms	28
A.2	Key API Endpoints	29
A.3	Project Timeline	29

List of Figures

3.1	High-level three-tier system architecture of Nikah.	9
3.2	Context-level diagram (DFD-0).	10
3.3	Compatibility matching algorithm flowchart.	13
5.1	Home page hero with branding and primary calls to action.	21
5.2	Biodata details page with contact-request and favourite actions.	21
5.3	Biodatas listing with filter sidebar and paginated card grid.	21
5.4	User dashboard overview with summary cards and activity feed.	22
5.5	Compatibility matches with percentage scores and breakdowns.	22
5.6	Administrator dashboard with analytics charts and stat cards.	23
5.7	Manage users table with role and premium controls.	23

List of Tables

2.1	Comparative analysis of existing matrimony platforms.	4
3.1	Front-end technologies and their roles.	6
3.2	Back-end technologies and their roles.	7
3.3	MongoDB collections (Mongoose models).	7
3.4	Development and deployment tools.	8
3.5	Sprint plan and deliverables.	8
3.6	Use-case summary by actor.	9
3.7	Selected schema field detail.	12
5.1	Hardware requirements.	20

Chapter 1

Introduction

1.1 Problem Specification

Marriage occupies a sacred position in Islam. The Prophet Muhammad (peace be upon him) is reported to have said, “*When a person marries, he has fulfilled half of his religion*” (reported by al-Bayhaqi). Today the majority of people turn to the internet when searching for a life partner, yet the platforms that are widely available do not serve the Bangladeshi Muslim community well.

Leading matrimony services such as **Shaadi.com** and **Muslima.com** address a global audience and are not built around Islamic values; family involvement, privacy in accordance with the principle of *hijab*, and religious compatibility are largely absent. Personal contact information is frequently exposed without proper consent, creating genuine risks of harassment and data misuse. Language is a further barrier: most Bangladeshi users prefer Bangla, yet the dominant platforms operate in English. Pricing is prohibitive, mobile optimization is poor despite mobile being the primary means of internet access in Bangladesh, fake profiles are common, and there is no reliable way to verify authenticity or to report suspicious activity.

These gaps motivated the construction of a platform that is modern, secure, Islamically aligned and built specifically for Bangladeshi users — the gap that **Nikah** is designed to fill.

1.2 Objectives

The principal objectives of the project were:

1. To build a complete matrimony platform with a React front end, a Node.js/Express back end and a MongoDB database.
2. To design features that respect Islamic values — privacy, family involvement and halal communication.
3. To allow members to create and manage detailed biodata profiles with personal, family and partner-preference information.
4. To provide rich search filters and a compatibility matching engine based on age, height, division and occupation.
5. To secure the platform with JWT authentication, role-based access control and contact-information gating.

6. To keep the architecture modular so that it can scale as the user base grows.
7. To support both English and Bangla through internationalization.
8. To deliver a professional administration dashboard with analytics, a premium subscription model, user-to-user messaging and a fully responsive, dark-mode-capable interface.

1.3 Scope

The scope of the project encompasses the complete design, development and cloud deployment of a web-based matrimony platform. In scope are: user registration and authentication (including a separate, secure administrator login); biodata lifecycle management; filtered search and a compatibility matching engine; contact-request workflow with payment gating; favourites, messaging and notifications; profile-view tracking and recently-viewed history; premium subscriptions and success stories; a reporting system; and a analytics-rich admin dashboard. Bilingual (English/Bangla) support, a dark theme, responsive design and cloud deployment (Vercel, Netlify, MongoDB Atlas) are also in scope.

Out of scope for this phase are native mobile applications, real-time WebSocket-based chat, live payment processing (the Stripe integration runs in demonstration mode), machine-learning-based recommendations, and identity verification — these are discussed under future work.

1.4 Flow of the Project

Development was organized into six phases:

Phase 1 — Planning & Design (Weeks 1–2): Requirements gathering, use-case modelling, ER design, architecture planning and UI wireframing.

Phase 2 — Back-End Development (Weeks 3–5): MongoDB schema design with Mongoose, Express server bootstrap, JWT authentication, the environment-based administrator login, and CRUD endpoints for every collection.

Phase 3 — Front-End Development (Weeks 6–9): React/Vite scaffold, React Router configuration, Tailwind and shadcn/ui theming, Axios-based API integration, and construction of all pages.

Phase 4 — Feature Enhancement (Weeks 10–11): Messaging, notifications, the compatibility matching engine, profile-view tracking, recently-viewed history and premium subscription handling.

Phase 5 — Testing & QA (Weeks 12–13): API testing with Postman, UI testing, accessibility checks, bug fixing and the security hardening of administrator access.

Phase 6 — Deployment & Documentation (Weeks 14–15): Deployment of the API to Vercel and the front end to Netlify, environment-variable configuration, report writing and presentation preparation.

1.5 Organization of the Project Report

The remainder of this report is organized into six chapters. **Chapter 2** surveys existing systems and the supporting literature. **Chapter 3** presents the system analysis and design — the technology stack, the development model, the system architecture, the use-case, context and data-flow diagrams, the database schema, and the core algorithms. **Chapter 4** describes the implementation of the front end and back end and enumerates the functional modules. **Chapter 5** is a user manual covering system requirements and interface walkthroughs. **Chapter 6** concludes with a discussion of limitations and future work, followed by the references and appendices.

Chapter 2

Background and Related Works

2.1 Existing System Analysis

Before constructing Nikah, the team examined the platforms already available in order to understand where the gaps lay. Table 2.1 summarises the strengths and weaknesses of the most relevant competitors.

Table 2.1: Comparative analysis of existing matrimony platforms.

Platform	Strengths	Weaknesses
Shaadi.com	Large user base, global reach	Not Islam-specific, expensive, English-only
Muslima.com	Muslim audience, international	Outdated interface, limited Bangla, costly
BiyeKotha (BD)	Bangla support, local focus	Poor UI, limited features, not mobile-friendly
Muzz (Muzmatch)	Muslim-focused, modern, app-based	Heavy premium model, few free features
Tinder / Bumble	Modern interface, large user base	Not for marriage, no Islamic alignment

The recurring gaps identified were:

- No single platform combines a modern interface, Islamic values and Bangla support.
- None offers a structured compatibility matching algorithm.
- Administrator dashboards with analytics are absent across the board.
- Role-based access with a premium tier is not implemented.
- Real-time-style messaging and notification systems are missing.
- Profile-view tracking and recently-viewed history are not provided.

Nikah addresses every one of these gaps: it unifies a modern shadcn/ui and Tailwind interface with Islamic privacy controls, full Bangla support, a compatibility engine, an analytics dashboard, messaging, notifications and a freemium subscription model.

2.2 Supporting Literatures

The work draws on several bodies of knowledge — Islamic teachings on marriage, modern web architecture patterns, authentication frameworks, and the specific front-end and back-end technologies employed.

Islamic foundations. The platform respects the principle of *hijab* by gating contact information — it is revealed only after a verified, paid or premium request. Biodata captures the father’s and mother’s names so that families are involved. Communication is structured and verifiable, and the hadith on marriage completing half of one’s faith (al-Bayhaqi) is the underlying motivation.

Architecture. The system follows a classical three-tier pattern: a React single-page presentation layer, an Express REST application layer with modular routers, and a MongoDB data layer managed by Mongoose.

Authentication and authorization. Members authenticate through Firebase (email/password and Google OAuth) and exchange the verified identity for a server-issued JWT with a seven-day expiry. The administrator authenticates separately against environment-configured credentials. A three-tier RBAC scheme (user, premium, admin) is enforced by Express middleware.

Front-end stack. React 18 was chosen for its ecosystem; Vite provides fast builds and hot-module replacement; Tailwind CSS with `darkMode: 'class'` and the shadcn/ui component system (built on Radix UI primitives) deliver an accessible, consistent and themeable UI; TanStack React Query manages server state with caching; and Framer Motion supplies animation.

Back-end stack. Node.js and Express 5 form the server; Mongoose 9 provides the ODM layer; Stripe handles payments (in demonstration mode); and `bcryptjs` and `jsonwebtoken` provide password hashing and token signing/verification respectively.

Database and deployment. MongoDB Atlas offers a free, cloud-hosted tier with auto-scaling; thirteen collections were designed with indexes on frequently queried fields. The API runs as serverless functions on Vercel and the front end is served by Netlify with continuous deployment from Git.

2.3 Problem Statement and Research Gap

The specific problem this project solves can be stated as follows: *Bangladeshi Muslims lack a single, trustworthy online platform that simultaneously (a) respects Islamic privacy and family involvement, (b) speaks their language, (c) matches compatibility systematically rather than only by free-text search, (d) protects contact information behind verified access, and (e) gives operators meaningful administrative oversight.* Existing solutions each satisfy at most one or two of these requirements. Nikah is the integrated response to that research gap.

Chapter 3

System Analysis and Design

3.1 Technology and Tools

3.1.1 Front-End Stack

Table 3.1: Front-end technologies and their roles.

Technology	Version	Role
React.js	18.3.1	UI framework
Vite	7.2.4	Build tool and dev server
React Router DOM	7.10.1	Client-side routing
Tailwind CSS	3.4.17	Utility-first styling
shadcn/ui (new-york)	—	Component system (Radix UI based)
Radix UI primitives	16 pkgs	Accessible headless components
TanStack React Query	5.90.12	Server-state caching and fetching
Axios	1.13.2	HTTP client
Firebase	12.6.0	User authentication (email/Google)
Framer Motion	12.23.25	Animation
Recharts	3.5.1	Admin analytics charts
Stripe React/JS	5.4 / 8.5	Payment UI (demo)
React Helmet Async	2.0.5	Per-page SEO meta tags
React Hot Toast	2.6.0	Toast notifications
SweetAlert2	11.26.3	Confirmation dialogs
Lucide & React Icons	5.x	Iconography
Noto Sans Bengali	5.2.11	Bengali typeface

3.1.2 Back-End Stack

Table 3.2: Back-end technologies and their roles.

Technology	Version	Role
Node.js	24.10.0	JavaScript runtime
Express.js	5.2.1	Web framework / REST API
Mongoose	9.0.0	MongoDB ODM
jsonwebtoken	9.0.3	JWT creation and verification
bcryptjs	3.0.3	Password hashing (admin credentials)
cors	2.8.5	Cross-origin request control
cookie-parser	1.4.7	Cookie handling
stripe	20.0.0	Payment processing (demo)
dotenv	17.2.3	Environment-variable management

3.1.3 Database

The database is hosted on **MongoDB Atlas** (M0 free tier) and contains thirteen collections, listed in Table 3.3.

Table 3.3: MongoDB collections (Mongoose models).

Model	Purpose
User	Account identity, role and premium status
Biodata	Matrimonial profile (personal, family, preferences)
ContactRequest	Paid/premium request for another member’s contact info
ContactMessage	Public “Contact Us” form submissions
Favorite	Bookmarked biodata records
Match	Computed compatibility pairings and scores
Message	User-to-user private messages
Notification	In-app notifications (six types)
ProfileView	“Who viewed me” tracking
RecentlyViewed	Per-user recently viewed biodata history
Report	Reports against profiles (five reasons)
Subscription	Premium plan subscriptions and lifecycle
SuccessStory	“Got Married” submissions

3.1.4 Tools Used

Table 3.4: Development and deployment tools.

Tool	Purpose
Git + GitHub	Version control and collaboration
VS Code	Code editor
Postman	API testing
MongoDB Atlas dashboard	Database management
Vercel	Back-end (serverless) deployment
Netlify	Front-end deployment
Firebase Console	Authentication management
Figma	UI wireframing

3.2 Development Model

The project followed an **Agile Scrum** methodology. Each sprint lasted one week: tasks were planned and prioritised at the start of the sprint, progress was reviewed regularly, and a working increment was delivered at the end. This suited the project because requirements continued to evolve as development deepened; the weekly cadence forced incremental delivery of working features rather than a single late integration. Table 3.5 summarises the phases.

Table 3.5: Sprint plan and deliverables.

Phase	Duration	Deliverable
Planning & Design	Weeks 1–2	Requirements, ER diagram, architecture, wireframes
Back-End Development	Weeks 3–5	Models, routers, auth, middleware
Front-End Development	Weeks 6–9	Components, pages, routing, API integration
Feature Enhancement	Weeks 10–11	Messaging, matching, notifications, subscriptions
Testing & QA	Weeks 12–13	API/UI tests, bug fixes, security hardening
Deployment & Docs	Weeks 14–15	Live deployment, report, presentation

3.3 System Architecture

The system is organized into three tiers plus two external services, shown in Figure 3.1. The front end is a React single-page application served by Netlify that communicates with the Express REST API (deployed as serverless functions on Vercel) over HTTPS using Axios. The API performs business logic, authentication, validation and database operations against MongoDB Atlas. Firebase handles member authentication outside the main request flow, and Stripe handles payment intents (in demonstration mode).

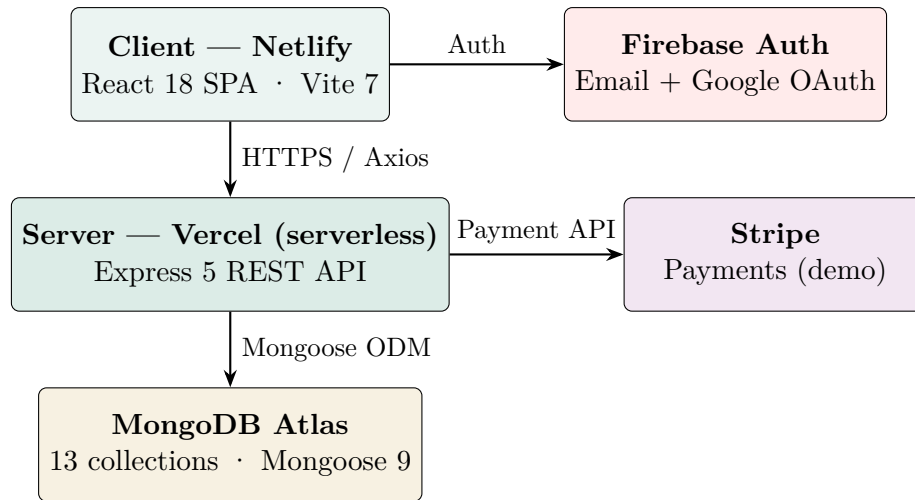


Figure 3.1: High-level three-tier system architecture of Nikah.

3.4 Use-Case Analysis

Four actors interact with the system. Their principal use cases are summarised in Table 3.6; the inheritance relationship is that a *premium user* inherits every capability of a *registered user* and adds premium privileges.

Table 3.6: Use-case summary by actor.

Actor	Representative use cases
Guest	Register; log in; browse the home page; view success stories; use the contact form.
Registered user	Create / edit / view own biodata; search with filters; view biodata details; add or remove favourites; request contact information; send and receive messages; read notifications; view compatibility matches; see who viewed their profile; browse recently-viewed profiles; view an activity feed; report a profile; compare biodatas; submit a success story; toggle theme and language.
Premium user	Everything a registered user can do, plus: view contact information without a per-request payment; unlimited contact requests; highlighted profile.
Administrator	View the analytics dashboard; manage users (search, promote/demote, grant or revoke premium); approve premium requests; approve contact requests; moderate success stories; read contact-form messages; handle reported profiles.

3.5 Context-Level Diagram (DFD-0)

The context diagram (Figure 3.2) shows the system as a single process exchanging flows with its external entities: guests and members, the Firebase authentication service, the MongoDB database and the Stripe payment service.

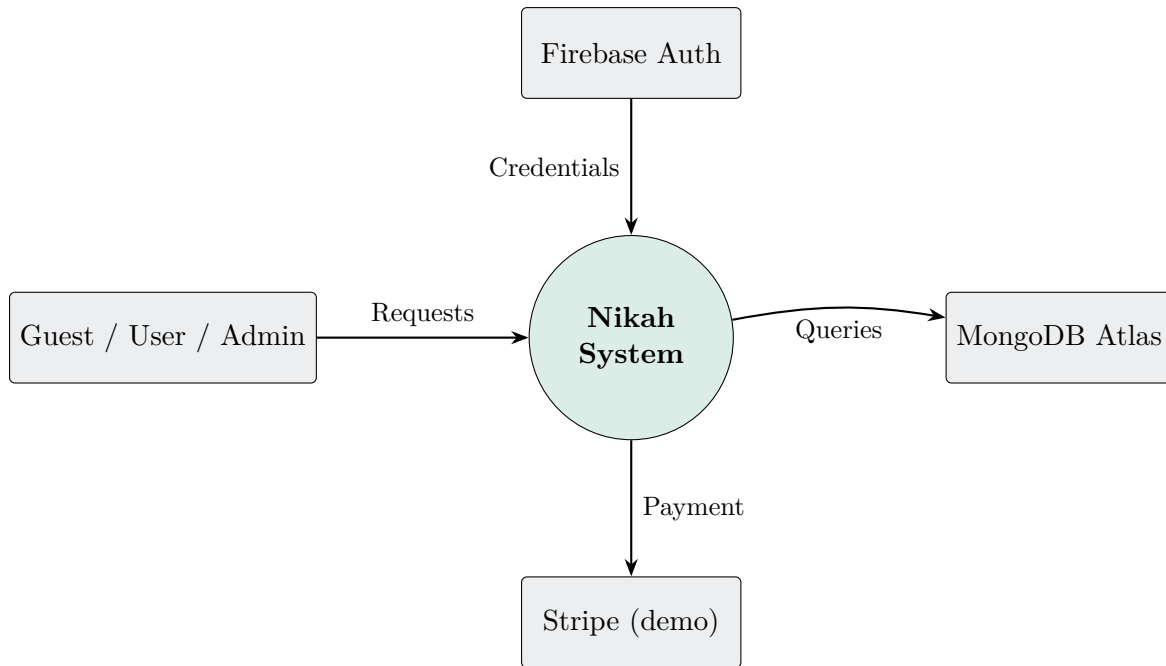
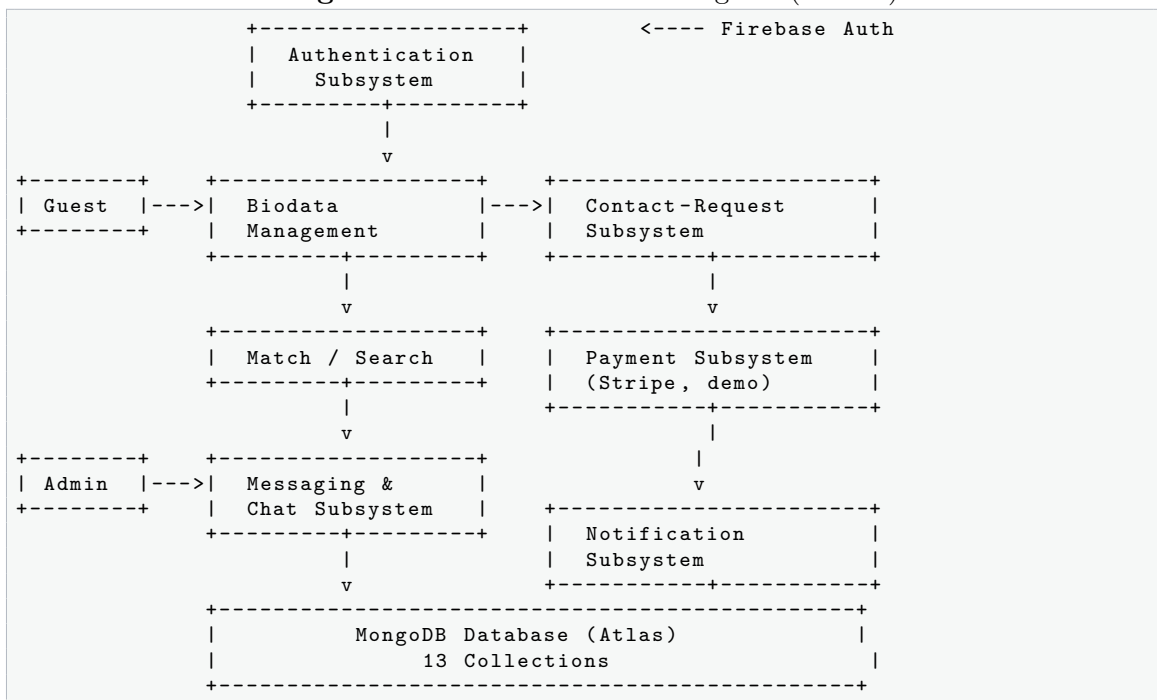


Figure 3.2: Context-level diagram (DFD-0).

3.6 Data-Flow Diagram (DFD-1)

The first-level decomposition (Listing 3.1) breaks the system into six cooperating sub-systems — authentication, biodata management, contact requests, payment, messaging and notifications — all of which read from and write to the shared MongoDB database.

Listing 3.1: First-level data-flow diagram (DFD-1).



3.7 Database Schema

The entity relationships among the thirteen collections are shown in Listing 3.2. Indexes were added on frequently queried fields — `email`, `biodataId`, `biodataType`, `permanentDivision` and `age` — to keep query latency low as the data grows. Selected field-level detail is given in Table 3.7.

Listing 3.2: Entity-relationship overview of the Nikah database.

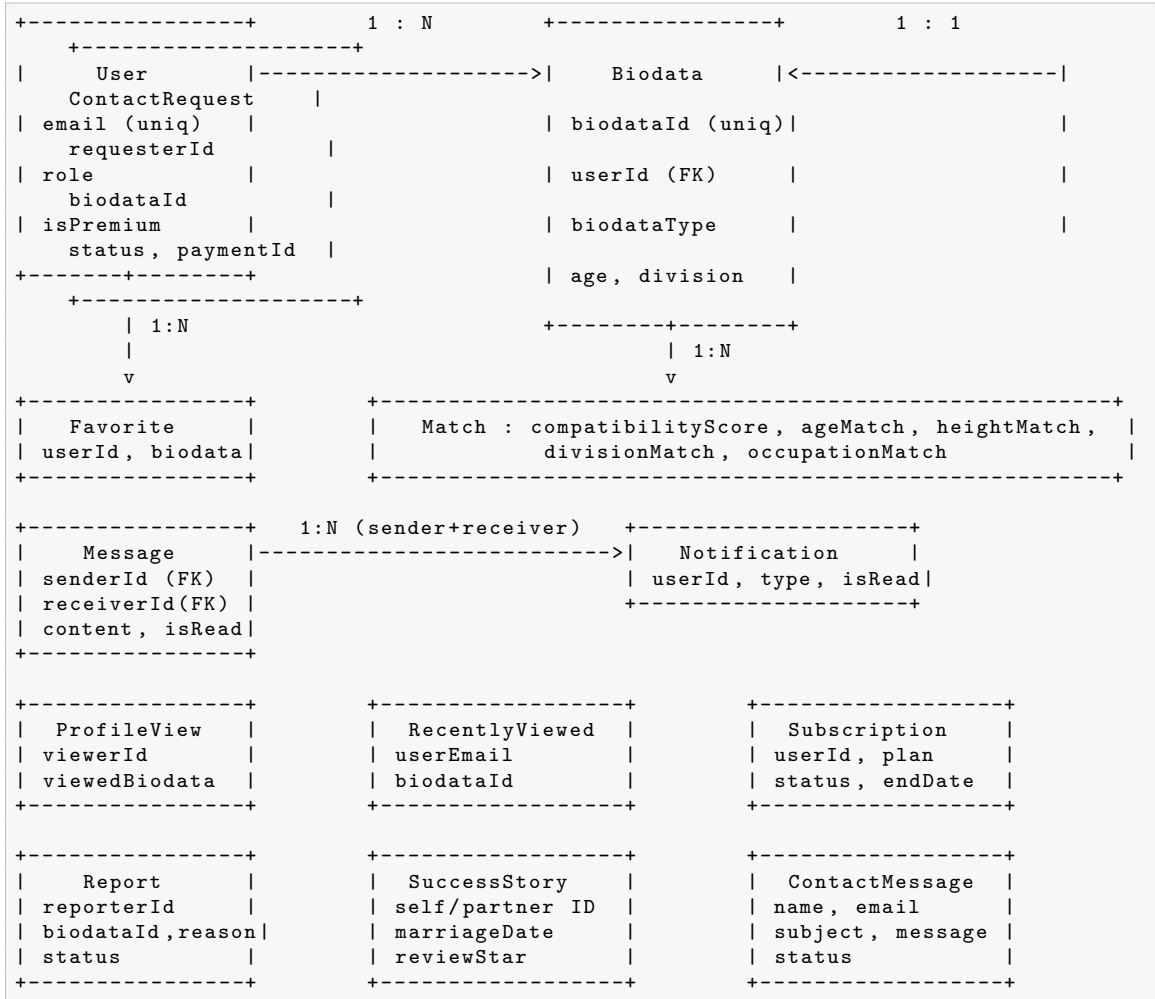


Table 3.7: Selected schema field detail.

Model	Key field	Notes
User	role	enum ['user', 'admin']; isPremium boolean; premiumRequestStatus enum
Biodata	biodataId	unique auto-incrementing number; biodataType Male/Female; 7 divisions; occupation & race enums
ContactRequest	status	enum ['pending', 'approved']; paymentId required
Match	compatibilityScore	numeric 0–100; matchDetails booleans; status enum
Message	content	senderId/receiverId; isRead flag; indexed by email
Notification	type	six types (contact_request, contact_approved, premium_approved, new_message, profile_viewed, system)
Report	reason	five reasons; status pending/reviewed/resolved/dismissed
Subscription	plan	basic/premium/gold; payment method incl. bKash/Nagad/Rocket; status lifecycle
SuccessStory	marriageDate	1–5 star rating; pre('save') hook auto-fills male/female biodata IDs

3.8 Algorithms and Flowcharts

3.8.1 Compatibility Matching

The matching algorithm compares a member’s stored partner preferences against every opposite-gender candidate and produces a percentage score in the range 0–100. Four criteria are evaluated: age range, height, division and occupation. The score is the ratio of matched criteria to the number of applicable criteria, expressed as a percentage, and a per-criterion breakdown is returned so that members can see exactly what matched. Up to fifty candidates are scored and the top twenty are returned, sorted in descending order. The flowchart is shown in Figure 3.3, and the implementation in Listing 3.3.

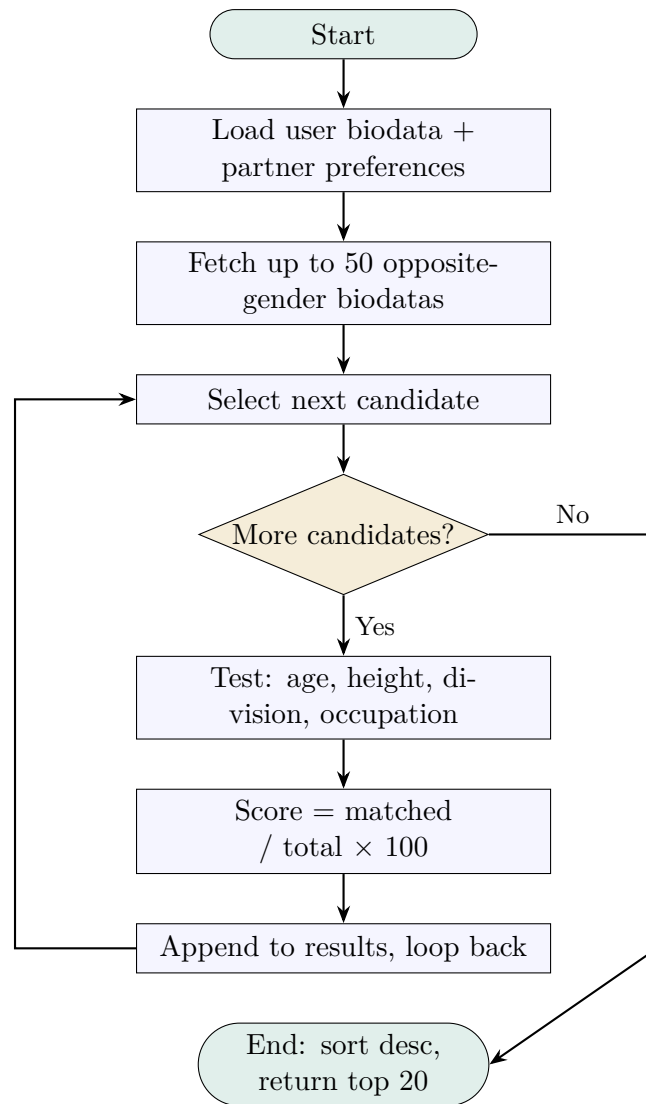


Figure 3.3: Compatibility matching algorithm flowchart.

Listing 3.3: Compatibility scoring (server, simplified).

```
function calculateCompatibility(myBio, target) {
  let score = 0, total = 0;
  const details = { ageMatch:false, heightMatch:false,
                    divisionMatch:false, occupationMatch:false };
  // 1. Age within the expected "min-max" range
  if (myBio.expectedPartnerAge && target.age) {
    total++;
    const [min, max] = myBio.expectedPartnerAge.split('-').map(
      Number);
    if (target.age >= min && target.age <= max) { score++; details.
      ageMatch = true; }
  }
  // 2. Exact height match
  if (myBio.expectedPartnerHeight && target.height) {
    total++;
    if (myBio.expectedPartnerHeight === target.height) { score++;
      details.heightMatch = true; }
  }
  // 3. Same permanent division
  if (myBio.permanentDivision && target.permanentDivision) {
    total++;
    if (myBio.permanentDivision === target.permanentDivision) {
      score++; details.divisionMatch = true; }
  }
  // 4. Same occupation
  if (myBio.occupation && target.occupation) {
    total++;
    if (myBio.occupation === target.occupation) { score++; details.
      occupationMatch = true; }
  }
  return { score: total > 0 ? Math.round((score / total) * 100) : 0,
    details };
}
```

3.8.2 Authentication Flow

Two authentication paths exist. **Members** authenticate through Firebase (email/password or Google) and the verified identity is exchanged for a server JWT at `POST /api/auth/jwt`. **Administrators** authenticate directly against environment-configured credentials at `POST /api/auth/admin-login`, entirely outside Firebase. The flow is shown in Listing 3.4.

Listing 3.4: Authentication flow (member and administrator paths).

```

Visit site -> Login / Register
|
+--[ MEMBER ]--> Email/Password or Google --> Firebase validates
|
|               |               |
|               | (fail) -> show error   (ok) -> POST /api/auth/jwt
|               |
|               |               |
|               |               | Server upserts User (role=user)
|               |               |
|               |               | Signs JWT (7-day expiry)
|               |               |
|               +<----- token stored, redirect ----->
|
+--[ ADMIN  ]--> /admin-login -> POST /api/auth/admin-login (email+password)
|
| Server checks ADMIN_EMAIL + ADMIN_PASSWORD (bcrypt or plain)
|
| (fail) -> 401 Invalid admin credentials
| (ok)   -> upserts User (role=admin, isPremium=true),
|           signs JWT with adminLogin claim (7-day expiry)
|
| Client builds synthetic admin user, sets isAdmin=true,
| redirects to /dashboard/admin

```

3.8.3 Contact-Request Workflow

Listing 3.5: Contact-information request workflow.

```

Member views biodata -> clicks "Request Contact Information"
|
+--[ Premium member ]--> create request directly
|
+--[ Regular member ]--> redirect to Stripe checkout
|
|                           |
|                           | payment completed (demo)
|                           |
|                           | create ContactRequest with paymentId
|
|
v
Status: pending -> administrator notified -> administrator approves
|
v
Member can now view the contact information

```

Chapter 4

Implementation

4.1 Front-End / Interface Design

The front end is a React 18 single-page application built with Vite 7. It is organized into **pages** (route-level components), **layouts** (shared `MainLayout` with navbar/footer and `DashboardLayout` with sidebar), and **reusable components**, including twenty-four shadcn/ui primitives under `components/ui`.

Design system. The UI is built on **shadcn/ui** (new-york style, JSX, slate base) atop **Radix UI** primitives and styled with **Tailwind CSS** in `darkMode: 'class'` mode. The brand tokens are a custom **emerald-and-gold** palette — `--primary` deep emerald (HSL 160 84% 24%) with a champagne-gold accent — exposed as CSS custom properties and consumed through Tailwind. The **Inter** and **Outfit** typefaces carry Latin text and **Noto Sans Bengali** carries Bangla. Custom utilities (`gradient-islamic`, `gradient-gold`, `glass-nav`) and animations (`fade-in-up`, `shimmer`, `marquee`) reinforce the brand.

Routing. React Router 7 with `createBrowserRouter` defines thirty-four routes: twelve public routes under `MainLayout`, fourteen protected user-dashboard routes, seven administrator routes, and a catch-all `NotFound`. Guard components `PrivateRoute` and `AdminRoute` gate access based on authentication and role.

State management. Three React contexts manage global state: **AuthContext** (Firebase user or synthetic admin user, JWT, role, premium status, loading), **ThemeContext** (dark/light with `localStorage` persistence and system-preference fallback), and **LanguageContext** (English/Bangla with dot-path translation lookup over 707-line dictionaries). Provider nesting is `HelmetProvider` → `LanguageProvider` → `ThemeProvider` → `QueryClientProvider` → `AuthProvider`.

Server state. TanStack React Query manages all API data with a five-minute stale time, a single retry, and refetch-on-window-focus disabled.

API layer. A single Axios instance reads its base URL from `VITE_API_URL`. A request interceptor attaches the JWT from `localStorage` as a Bearer token and the active locale as an `Accept-Language` header; a response interceptor detects HTTP 401,

clears the token, and dispatches an 'unauthorized' window event that `AuthContext` listens for in order to log the user out globally.

UI libraries. Framer Motion for transitions, React Helmet Async for SEO, React Hot Toast and SweetAlert2 for feedback, and Recharts for the administrator charts.

4.2 Back-End

The back end is an Express 5 application (CommonJS). The entry point configures CORS with an origin allow-list (the configured client URL, localhost, any `*.netlify.app` and `*.vercel.app` origin, and origin-less requests such as mobile clients), enables JSON and cookie parsing, mounts seventeen routers under `/api/`, and registers a 404 handler and a centralized error handler. All business logic is implemented inline within the route handlers.

Middleware. Three functions control access: `verifyToken` extracts and verifies the JWT, lazily provisions a `User` document if the decoded email is new, and refreshes `req.user` with the current role and premium status; `verifyAdmin` treats the environment-configured `ADMIN_EMAIL` as the single source of truth for administrator identity and actively *downgrades* any database user whose email does not match but who holds `role:'admin'`; and `verifyPremium` rejects non-premium callers with HTTP 403.

Environment-based administrator login. This is a deliberate security design. The administrator does *not* use Firebase. At `POST /api/auth/admin-login` (Listing 4.1) the server compares the supplied credentials against `ADMIN_EMAIL` and `ADMIN_PASSWORD`. The password is accepted either as a `bcrypt` hash *or* as plain text (dual-mode compare), the email comparison is case-insensitive, and on success the server upserts a `User` with `role:'admin'` and `isPremium:true` and issues a seven-day JWT carrying a custom `adminLogin:true` claim. Because the client sets `isAdmin` directly from this response, the administrator can enter `/dashboard/admin/*` without ever touching Firebase.

Listing 4.1: Environment-based administrator login (server, simplified).

```

router.post('/admin-login', async (req, res) => {
  const { email, password } = req.body;
  const adminEmail = (process.env.ADMIN_EMAIL || '').trim().
    toLowerCase();
  const adminPassword = process.env.ADMIN_PASSWORD || '';
  if (!adminEmail || !adminPassword)
    return res.status(503).json({ message: 'Admin login not
      configured.' });
  if (email.trim().toLowerCase() !== adminEmail)
    return res.status(401).json({ message: 'Invalid admin
      credentials.' });
  const ok = await bcrypt.compare(password, adminPassword) //
    hashed
    || password === adminPassword; // or
    plain
  if (!ok) return res.status(401).json({ message: 'Invalid admin
    credentials.' });
  // upsert admin user, then sign a 7-day JWT with an adminLogin
  // claim
  const token = jwt.sign(
    { email, name: 'Admin', role: 'admin', isPremium: true,
      adminLogin: true },
    process.env.JWT_SECRET, { expiresIn: '7d' });
  res.json({ token, user: { /* ... */ } });
});

```

Route modules. The seventeen routers expose seventy handlers across authentication, biodata, contact requests, contact messages, favourites, matches, messages, notifications, payments, profile views, recently-viewed records, reports, statistics, subscriptions, success stories, analytics and administration. Selected endpoints are listed in Appendix A.2.

Database models. Thirteen Mongoose schemas (Table 3.3) carry field validation (required fields, enums, min/max values), timestamps, and indexes on hot fields. The Biodata model uses a unique auto-incrementing `biodataId`, and the SuccessStory model uses a `pre('save')` hook to auto-classify the male/female biodata IDs.

Security. JWTs expire after seven days; administrator identity is pinned to a single environment variable and stale administrator roles are actively demoted; contact information is hidden from non-premium users who do not own the biodata; CORS validates origins; and all secrets live in environment variables. The `.env` file is git-ignored.

4.3 Modules and Features

The following fifteen functional modules were implemented:

1. **Authentication** — Firebase email/password and Google for members, plus a separate secure, environment-based administrator login.

2. **Biodata management** — create and update biodata with sixteen or more fields, auto-incrementing IDs and automatic age calculation from date of birth.
3. **Search and discovery** — filter by gender, division, age range and occupation, with pagination (twenty per page) and sort options.
4. **Contact-request system** — request contact info with a Stripe payment for non-premium members, an administrator approval workflow, and a premium bypass.
5. **Favourites** — add and remove bookmarks with duplicate prevention.
6. **Messaging** — inbox, sent and conversation threading with unread tracking and notification generation.
7. **Notifications** — six notification types with unread badge, mark-as-read, mark-all-read and delete.
8. **Compatibility matching** — 0–100 percentage scoring across four criteria with a per-criterion breakdown and a mutual-score view.
9. **Profile-view tracking** — records views with self-view and one-hour duplicate suppression, and reports total and unique viewers.
10. **Recently viewed** — a per-user history of recently browsed biodata, kept in sync between `localStorage` and the server and automatically trimmed to twenty entries.
11. **Premium subscription** — three plans (Basic free, Premium, and Gold) with Stripe payment, contact-info access and profile highlighting.
12. **Reporting** — report profiles for five reasons, with an administrator review and resolve/dismiss workflow.
13. **Success stories** — married members submit a story with partner ID, couple photo, marriage date, rating and text, moderated by an administrator.
14. **Administration dashboard** — analytics charts for user growth, gender, location and age distribution, plus user management, premium and contact approvals, content moderation and revenue.
15. **Internationalization and theming** — English/Bangla switching persisted to storage and sent as a request header, plus a class-based dark/light theme with a system-preference fallback.

Chapter 5

User Manual

5.1 System Requirements

5.1.1 Hardware Requirements

Table 5.1: Hardware requirements.

Component	Minimum	Recommended
Processor	Intel Core i3 / AMD Ryzen 3	Intel Core i5 / AMD Ryzen 5
RAM	4 GB	8 GB
Storage	1 GB free	5 GB free
Display	1366×768	1920×1080
Internet	2 Mbps	10 Mbps
Device	Desktop, laptop, tablet, phone	Any modern-browser device

5.1.2 Software Requirements

For users. Windows 10+, macOS 11+, Linux, Android 10+ or iOS 14+; Chrome 90+, Firefox 90+, Safari 14+ or Edge 90+; JavaScript enabled; an internet connection.

For local development. Windows 10+, macOS 11+ or Ubuntu 20.04+; Node.js v18+ (v24.10.0 recommended); npm v9+; a MongoDB Atlas account (free tier) or MongoDB v6.0+ locally; VS Code; Git v2.30+; and a Firebase Console project with Authentication enabled.

5.2 User Interfaces

5.2.1 Panel A — Public Pages

The home page presents a hero with calls to action (“Get Started” and “Browse Biodatas”), featured premium profiles, an animated statistics counter, a “How It Works” section and a success-story gallery with ratings, all wrapped by a glass-style navbar and a footer with a newsletter form. The **biodatas listing** page offers a filter sidebar (biodata type, division, age range, occupation) over a paginated grid of cards; the **biodata details** page exposes contact-request, favourite and report actions; the **login**

and **register** pages offer Firebase email/password and Google; the dedicated **admin login** page is a separate, server-side credential form. Static pages cover About, Contact (with a form), Privacy and Terms, and a success-stories gallery and a biodata comparison page are also available. Representative interfaces are illustrated in Figures 5.1–5.3.

[Insert screenshot — Home / hero section]

Figure 5.1: Home page hero with branding and primary calls to action.

[Insert screenshot — Biodata details page]

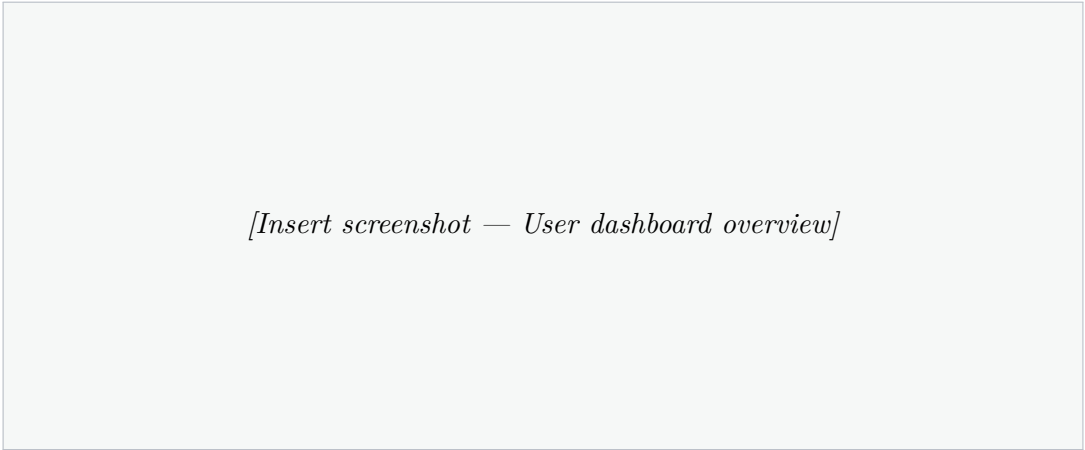
Figure 5.2: Biodata details page with contact-request and favourite actions.

[Insert screenshot — Biodatas listing page]

Figure 5.3: Biodatas listing with filter sidebar and paginated card grid.

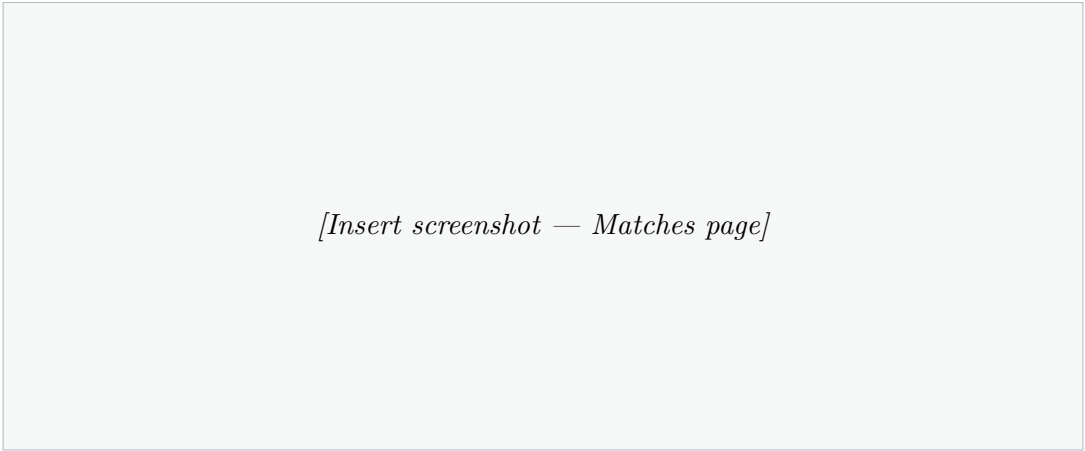
5.2.2 Panel B — User Dashboard

The dashboard provides an overview (biodata status, unread messages, favourites, profile views, activity feed and quick actions); **edit / view biodata** (with auto-calculated age and all seven divisions); **edit biodata** (full form with sixteen or more fields); **messages** (inbox, sent, conversation); **notifications**; **matches** (cards with 0–100 scores and breakdowns); **favourites**; **contact requests**; **profile views**; **recently viewed**; **settings** (theme, language, account); and **Got Married** (success-story submission).



[Insert screenshot — User dashboard overview]

Figure 5.4: User dashboard overview with summary cards and activity feed.

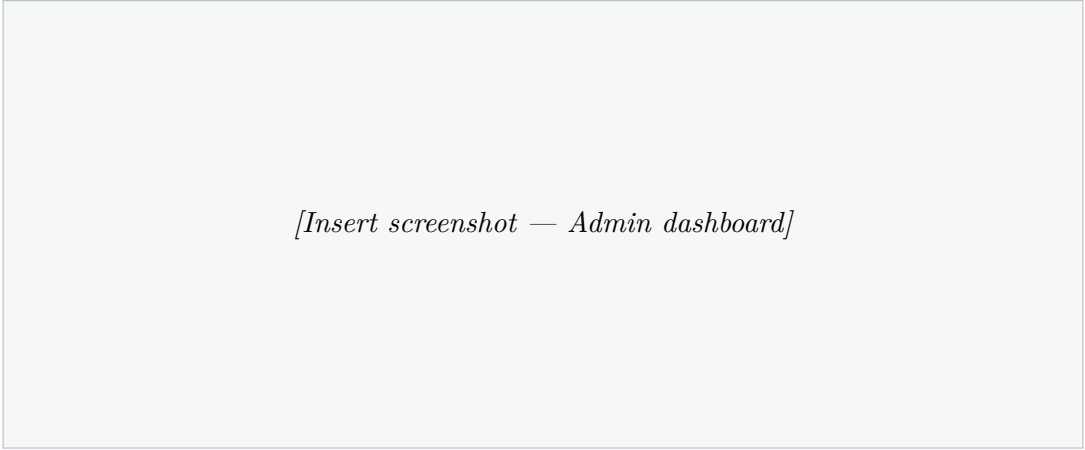


[Insert screenshot — Matches page]

Figure 5.5: Compatibility matches with percentage scores and breakdowns.

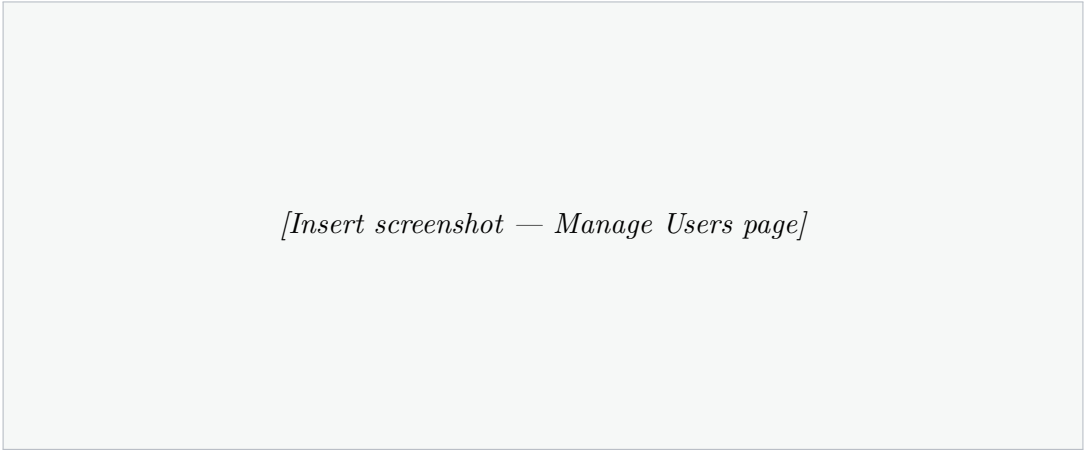
5.2.3 Panel C — Admin Panel

The administrator dashboard shows stat cards, an area chart for thirty-day user growth, a pie chart for gender distribution, a bar chart for location statistics, an age-distribution chart, and a recent-activity table. Further tabs cover **manage users** (search, promote/demote, grant or revoke premium), **approved premium**, **approved contacts**, **contact messages**, **success stories** moderation, and **reported profiles**.



[Insert screenshot — Admin dashboard]

Figure 5.6: Administrator dashboard with analytics charts and stat cards.



[Insert screenshot — Manage Users page]

Figure 5.7: Manage users table with role and premium controls.

5.2.4 Login Credentials

Test member (illustrative) — e-mail `user@test.com`, password `User@123`. **Administrator** — the administrator logs in at `/admin-login` using the e-mail and password configured in the server's `ADMIN_EMAIL` and `ADMIN_PASSWORD` environment variables; registration with the administrator e-mail through the public form is not permitted.

5.3 Getting Started

Clone the repository, run `npm install` in both `client/` and `server/`, copy `server/.env.example` to `server/.env` and fill in `MONGODB_URI`, `JWT_SECRET`, `CLIENT_URL`, `ADMIN_EMAIL`, `ADMIN_PASSWORD` and (optionally) `STRIPE_SECRET_KEY`, then start the API (`npm run dev`) and the client (`npm run dev`).

Chapter 6

Conclusion

6.1 Conclusion

Nikah matured into a full-featured matrimony platform that addresses real gaps in the market for Bangladeshi Muslims. The final system comprises thirteen database collections, seventy API route handlers across seventeen modules, and fifteen functional modules spanning authentication, biodata management, search, messaging, matching, payments and administration. The front end is responsive, bilingual and dark-mode capable, built on a shadcn/ui emerald-and-gold design system, and administrators are equipped with a proper analytics dashboard.

Security was treated as a first-class concern. JWT authentication with role-based access control, contact-information gating, a dedicated environment-based administrator login that is isolated from Firebase, an origin-validated CORS policy, and active demotion of stale administrator roles combine to keep the platform locked down. The deployment is live — the API on Vercel, the front end on Netlify and the database on MongoDB Atlas, all on free tiers. Just as importantly, the features were designed around Islamic values: privacy through contact gating, family involvement through parent-name fields, and verifiable, structured communication that supports halal matchmaking.

6.2 Limitations

1. **No native mobile application.** The web version is responsive, but native iOS and Android apps with push notifications would improve reach.
2. **Payments run in demonstration mode.** Stripe is wired up but does not process real transactions, and local gateways such as bKash, Nagad and Rocket are not integrated.
3. **No real-time transport.** Messaging and notifications operate on a request-response cycle; WebSockets would make chat instantaneous.
4. **Rule-based matching.** The engine compares four criteria with equal weight; machine learning could produce more nuanced recommendations.
5. **No identity verification.** The platform trusts user-entered data; a verification system would build trust.

6. **Single administrator identity.** Administrator access is pinned to one environment-configured account; a permission hierarchy would be more flexible.
7. **Two languages only.** English and Bangla are covered; Arabic and Urdu would broaden reach.
8. **No guardian (wali) workflow.** Guardian involvement, central to Islamic marriage, is not yet modelled.

6.3 Future Works

Priority directions for future work include native mobile applications (React Native); AI-powered matchmaking that learns from interaction patterns; WebRTC-based voice and video calling for premium members; background verification (identity, education, employment); additional languages; real payment integration with Stripe, bKash, Nagad and Rocket; a guardian (wali) system that can review messages and approve matches; advanced analytics and match-success prediction; wedding-planning tools (vendor directories, checklists, budgets); community features (counselling resources, scholar consultations); real-time chat with typing indicators and read receipts; and verified-profile badges.

References

- [1] React. “React Documentation.” <https://react.dev/>, 2026.
- [2] Vite. “Vite — Next Generation Frontend Tooling.” <https://vite.dev/>, 2026.
- [3] React Router. “React Router Documentation.” <https://reactrouter.com/>, 2026.
- [4] Tailwind Labs. “Tailwind CSS Documentation.” <https://tailwindcss.com/docs>, 2026.
- [5] shadcn. “shadcn/ui — Build Your Component Library.” <https://ui.shadcn.com/>, 2026.
- [6] Radix UI. “Radix Primitives.” <https://www.radix-ui.com/>, 2026.
- [7] TanStack. “TanStack Query Documentation.” <https://tanstack.com/query/>, 2026.
- [8] OpenJS Foundation. “Node.js.” <https://nodejs.org/>, 2026.
- [9] OpenJS Foundation. “Express — Fast, unopinionated, minimalist web framework for Node.js.” <https://expressjs.com/>, 2026.
- [10] MongoDB, Inc. “MongoDB Documentation.” <https://www.mongodb.com/docs/>, 2026.
- [11] Mongoose. “Mongoose ODM Documentation.” <https://mongoosejs.com/docs/>, 2026.
- [12] Google. “Firebase Authentication.” <https://firebase.google.com/docs/auth>, 2026.
- [13] IETF. “JSON Web Tokens.” <https://jwt.io/>, 2026.
- [14] Stripe. “Stripe API Documentation.” <https://stripe.com/docs/api>, 2026.
- [15] Axios. “Axios HTTP Client.” <https://axios-http.com/>, 2026.
- [16] Framer. “Framer Motion.” <https://www.framer.com/motion/>, 2026.
- [17] Recharts. “Recharts — composable charting library.” <https://recharts.org/>, 2026.
- [18] MongoDB, Inc. “MongoDB Atlas Documentation.” <https://www.mongodb.com/docs/atlas/>, 2026.

- [19] Vercel. “Vercel Deployment Documentation.” <https://vercel.com/docs>, 2026.
- [20] Netlify. “Netlify Documentation.” <https://docs.netlify.com/>, 2026.
- [21] dcodeIO. “bcrypt.js.” <https://github.com/dcodeIO/bcrypt.js>, 2026.
- [22] W3C. “Web Content Accessibility Guidelines (WCAG) 2.1.” <https://www.w3.org/WAI/WCAG21/quickref/>, 2026.
- [23] Sunnah.com. “Sahih al-Bukhari — Book of Marriage.” <https://sunnah.com/bukhari>, 2026.
- [24] al-Bayhaqi. *Hadith on marriage fulfilling half of one’s faith*.
- [25] Lucide. “Lucide Icons.” <https://lucide.dev/>, 2026.
- [26] SweetAlert2. “A beautiful, responsive, customizable replacement for JavaScript’s popup boxes.” <https://sweetalert2.github.io/>, 2026.
- [27] Google. “Noto Sans Bengali.” <https://fonts.google.com/noto>, 2026.
- [28] Lucidchart. “What is a Data Flow Diagram.” <https://www.lucidchart.com/pages/data-flow-diagram>, 2026.
- [29] Creately. “Use Case Diagram Tutorial.” <https://creately.com/guides/use-case-diagram-tutorial/>, 2026.
- [30] InterviewBit. “System Architecture Tutorial.” <https://www.interviewbit.com/blog/system-architecture/>, 2026.

Appendix A

Glossary and Supplementary Material

A.1 Glossary of Terms

Term	Meaning
Nikah	The Islamic marriage contract.
Biodata	A matrimonial profile with personal, family and preference details.
JWT	JSON Web Token — a compact token format for authentication.
RBAC	Role-Based Access Control.
REST API	A web API that follows REST conventions.
MERN	MongoDB, Express, React and Node.js.
ODM	Object Document Mapping between code objects and database documents.
i18n	Internationalization.
CORS	Cross-Origin Resource Sharing — a browser security mechanism.
Stripe	Online payment-processing platform.
Firebase	Google's application platform (used here for authentication).
Vercel	Cloud platform for serverless and static deployment.
Netlify	Cloud platform for front-end deployment.
WCAG	Web Content Accessibility Guidelines.
DFD	Data Flow Diagram.
ER Diagram	Entity-Relationship Diagram.
Hijab	Islamic concept of modesty and privacy.
Wali	Islamic marriage guardian.
Halal	Permissible under Islamic law.
bKash/Nagad/Rocket	Mobile financial services in Bangladesh.

A.2 Key API Endpoints

Method	Endpoint	Purpose	Auth
POST	/api/auth/jwt	Exchange Firebase identity for a server JWT	Public
POST	/api/auth/admin-login	Environment-based administrator login	Public
GET	/api/auth/me	Current user	Token
GET	/api/biodata	List biodata with filters	Public
GET	/api/biodata/:id	Get one biodata	Mixed
POST	/api/biodata	Create or update biodata	Token
POST	/api/contact-requests	Request contact information	Token
GET	/api/matches	Compatibility matches	Token
GET	/api/matches/with/:id	Mutual compatibility with a biodata	Token
GET	/api/messages/inbox	Received messages	Token
POST	/api/messages	Send a message	Token
GET	/api/notifications	Notifications	Token
POST	/api/favorites	Add a favourite	Token
GET	/api/recently-viewed	Recently viewed history	Token
POST	/api/subscriptions	Create a subscription	Token
POST	/api/payment/create-payment-intent	Create a Stripe payment intent	Token
GET	/api/stats/public	Public platform statistics	Public
GET	/api/analytics/stats	Administrator analytics	Admin
GET	/api/admin/users	List all users	Admin
PATCH	/api/admin/users/:id/make-admin	Promote a user to administrator	Admin

A.3 Project Timeline

